Veterinaria San Marcos

Especificación de Requisitos de Software (ERS V2) & Informe de Evaluación

DOCUMENTO OFICIAL

PROYECTO	MÓDULO	VERSIÓN	ESTADO TEST
Sistema Fullstack VET	Recepción & Reservas	2.0.0 (Evaluación P2)	10/10 PASS (100%)

1. Resumen Ejecutivo

El presente documento consolida la Especificación de Requisitos de Software Versión 2 (ERS V2) y el reporte técnico de pruebas unitarias para el **Módulo de Recepción y Gestión de Citas** de la Veterinaria San Marcos. El desarrollo comprende la integración entre la interfaz interactiva desarrollada en React/Vite, la capa de persistencia simulada con almacenamiento local (`localStorage`) y la suite automatizada de pruebas ejecutada sobre Karma y Jasmine.

2. Arquitectura de Software y Tecnologías

Capa del Frontend (React + Vite)

Componente modular `RecepcionistaDashboard.jsx` construido bajo arquitectura funcional de React con hooks (`useState`, `useEffect`). Proporciona renderizado reactivo e interfaz adaptable.

Capa de Datos (Persistencia Local)

Módulo reutilizable `js/admin/db.js` que centraliza las operaciones CRUD sobre la clave de almacenamiento `san_marcos_reservas`, garantizando persistencia sincrónica entre sesiones.

Test Runner (Karma v6.4.4)

Servidor de pruebas integrado con navegador ChromeHeadless / EdgeHeadless que automatiza la ejecución de especificaciones BDD/TDD en entornos de integración continua.

Framework de Pruebas (Jasmine + RTL)

Librería React Testing Library empleada para simular interacciones reales de usuario (eventos DOM, clics, renderizado de nodos) y aserciones rigurosas sobre la interfaz.

3. Alcance del Sistema

El módulo de recepción permite la atención expedita de pacientes clínicos veterinarios mediante el registro de mascotas, propietarios, horarios asignados, selección de servicios veterinarios y actualización del estado de las atenciones en tiempo real.

4. Matriz de Requisitos Funcionales (CRUD)

A continuación se especifican los requisitos funcionales implementados en la versión 2.0 del módulo de recepción, vinculando cada requerimiento a su función homóloga en la capa de datos `db.js`:

ID	NOMBRE REQUISITO	DESCRIPCIÓN OPERATIVA	FUNCIÓN ASOCIADA
RF-01	Creación de Cita	Permite agendar una nueva reserva capturando mascota, propietario, teléfono, fecha, hora y servicio.	`crearReserva()`
RF-02	Consulta y Lectura	Carga y recupera la lista completa de citas almacenadas para desplegarlas en el panel principal.	`getReservas()`
RF-03	Actualización / Estado	Modifica datos de la cita o cambia su estado operativo (Pendiente, Confirmada, Atendido).	`actualizarReserva()`
RF-04	Cancelación / Eliminación	Remueve permanentemente un registro de reserva del almacenamiento local tras confirmación.	`eliminarReserva()`
RF-05	Cálculo de Métricas	Calcula dinámicamente el conteo de citas de la fecha actual, pendientes y totales.	`getMetricasHoy()`

5. Modelo de Datos y Esquema de Almacenamiento

Los registros de reservas se estructuran como un arreglo de objetos JSON almacenados bajo la clave `san_marcos_reservas`. Cada objeto cumple con la especificación estructural mostrada en la siguiente tabla de atributos:

CAMPO	TIPO DATO	REQUERIDO	DESCRIPCIÓN / EJEMPLO
`id`	Number / Auto	Sí	Identificador único numérico asignado secuencialmente. Ex: 1
`mascota`	String	Sí	Nombre del paciente veterinario. Ex: `"Pepe"`, `"Luna"`
`dueno`	String	Sí	Nombre completo del propietario o tutor responsable. Ex: `"Ana García"`
`fecha / hora`	String (ISO)	Sí	Fecha de atención (`"YYYY-MM-DD"`) y hora programada (`"HH:mm"`).
`estado`	String (Enum)	Sí	Estado actual: `"Pendiente"`, `"Confirmada"` o `"Atendido"`.

6. Plan y Resultado de Pruebas Unitarias

La validación automatizada se ejecutó con la suite `RecepcionistaDashboard.test.jsx` sobre el test runner Karma. Se diseñaron 10 casos de prueba unitarios para verificar renderizado, interacción con el usuario, ejecución de callbacks y persistencia.

10	10	0	0.032s
PRUEBAS TOTALES	EXITOSAS (SUCCESS)	FALLIDAS	TIEMPO EJECUCIÓN

7. Matriz Detallada de Especificaciones de Prueba

Nº	NOMBRE / DESCRIPCIÓN DEL TEST	OBJETIVO DE VERIFICACIÓN	RESULTADO
01	Renderizado inicial sin errores	Comprobar el montaje del componente principal en el DOM.	SUCCESS
02	Despliegue de títulos principales	Verificar presencia de encabezados de navegación y bienvenida.	SUCCESS
03	Presencia de botón "+ Nueva Reserva"	Confirmar existencia del elemento interactivo de creación.	SUCCESS
04	Presencia de botón "Cerrar sesión"	Validar disponibilidad de la opción de salida de usuario.	SUCCESS
05	Respuesta al evento clic de reserva	Asegurar que el botón gatille la apertura del modal.	SUCCESS
06	Tarjeta métrica "CITAS DE HOY"	Verificar el indicador visual cuantitativo de citas diarias.	SUCCESS
07	Tarjeta métrica "PENDIENTES"	Validar el despliegue del conteo de solicitudes pendientes.	SUCCESS
08	Ejecución de prop `onLogout`	Comprobar la llamada a la función callback entregada por props.	SUCCESS
09	Lista de pacientes (Pepe y Luna)	Validar la lectura e inserción en DOM de registros mock.	SUCCESS
10	Lectura correcta desde LocalStorage	Verificar integración con la capa de datos almacenada.	SUCCESS

8. Análisis de Cobertura de Código (Code Coverage)

Mediante la herramienta Istanbul/Karma Coverage Reporter, se auditó la cobertura de sentencias, condicionales, funciones y líneas de la capa de persistencia `db.js`:

67.5%	33.3%	50.0%	68.6%
STATEMENTS (27/40)	BRANCHES (4/12)	FUNCTIONS (7/14)	LINES (24/35)

9. Evidencia Consolidada de Ejecución de Pruebas

A continuación se incluye la salida oficial capturada desde la consola de Karma Runner demostrando la ejecución sin errores de los 10 tests unitarios y la generación del reporte HTML:

```
[karma-server]: Karma v6.4.4 server started at http://localhost:9876/
[launcher]: Launching browsers ChromeHeadlessCI with concurrency unlimited
[launcher]: Starting browser ChromeHeadless
Edge 155.0.0.0 (Windows 10): Connected on socket UB27xA5r8863mWhzAAAA

Suite de Pruebas Unitarias - RecepcionistaDashboard
  ✔ 1. Debe renderizar el panel de recepción sin lanzar errores
  ✔ 2. Debe mostrar los títulos del panel de recepción
  ✔ 3. Debe visualizar el botón para crear una nueva reserva
  ✔ 4. Debe visualizar el botón para cerrar la sesión actual
  ✔ 5. Debe responder al evento clic en el botón de nueva reserva
  ✔ 6. Debe desplegar la tarjeta de métricas con las citas programadas de hoy
  ✔ 7. Debe desplegar la sección de citas o solicitudes pendientes
  ✔ 8. Debe ejecutar la función onLogout enviada por props
  ✔ 9. Debe listar en pantalla a los pacientes agendados como Pepe y Luna
  ✔ 10. Debe permitir la lectura de datos desde localStorage

Edge 155.0.0.0 (Windows 10): Executed 10 of 10 SUCCESS (0.032 secs)
```

10. Conclusión y Firma de Aprobación

El sistema cumple satisfactoriamente con la totalidad de los requerimientos funcionales y las pruebas unitarias exigidas en la pauta de evaluación parcial. La arquitectura es modular, escalable y apta para pasar a producción o fases posteriores de integración backend.

ROL / RESPONSABLE	ESTADO DE REVISIÓN	FECHA DE EMISIÓN
Desarrollador Fullstack Lead	APROBADO	18 de Octubre, 2026
QA & Testing Engineer	VERIFICADO (10/10)	18 de Octubre, 2026